

FieldMind — Multi-Agent Architecture Plan

Oct 1, 2026 · @Aarav Muni

The new design keeps every objective of the original plan. It changes how the work is cut up and where it runs, so that the CPU and NPU work at the same time and no single model call is large. On paper, a fault tick goes from about 19 s of one-after-another work, with the CPU idle about 99% of the time, to about 10 s with both units busy in parallel. The timing numbers are estimates from rates we have already measured and must be re-measured once one model is chosen.

Scope: what stays and what changes

Stays the same (from the Plan of Action):

- Same boiler, same six tags, same 30-second tick, same 30 episodes and answer keys.
- Same quality and system targets: Q1-Q7, S1-S7, constraints C1-C6, and energy per correct diagnosis as the number the optimisation drives.
- Same two core rules: the model never sees raw sensor numbers, and the model never has the last word.
- Same deterministic checks (L1), action gate (L6), case library, world model, advisory-only output, and the safe floor where everything still works with the model off.

Changes:

- One large diagnostician prompt becomes several small, single-purpose model calls, each under 1280 tokens.
- The model calls no longer have to finish inside the tick that started them. Every tick still publishes an assessment on time from the deterministic path. Model results arrive a few seconds later and update it.
- There are two model lanes, one on the NPU and one on the CPU, running at the same time. A scheduler decides which call goes where.
- Raw engineer notes and records are read once, when they arrive, by a separate agent. The diagnostician never reads raw text.
- Model answers are short and made of IDs (case IDs, fact IDs). The readable sentences are written by code.

Hardware constraint used throughout: one NPU and one CPU. Each can run one model call at a time, so at most two model calls run at once. The GPU is left out of this design; it can be added later as a third lane.

The two problems to solve

Problem 1: the CPU sits idle while the NPU works

In the current single agent, each step waits for the one before it. On a fault tick:

1. The CPU runs the deterministic checks, retrieval and belief update: about 0.2 s.
2. The NPU loads the model (3–4 s, because the model is reloaded on every call) and runs the diagnostician on a ~2484-token prompt: about 7.1 s.
3. The NPU loads the model again and runs the verifier: about 3.5 s of reload plus a few seconds of work.
4. The CPU runs the gate and memory write: a few milliseconds.

The CPU does about 0.2 s of work in a 30 s tick and waits for the rest. Making the NPU faster helps; making the CPU faster does nothing, because the CPU is never the bottleneck.

Problem 2: once prompts get short, writing the answer becomes the slow part

The time for one model call is roughly:

$$t_{\text{call}} \approx \frac{\text{prompt tokens}}{\text{prefill rate}} + \frac{\text{output tokens}}{\text{decode rate}} + \text{overhead}$$

On the NPU with llama.cpp we measured about 839 tok/s for prefill and about 11.7 tok/s for decode. Reading a 2484-token prompt takes about 3 s. Writing just 50 tokens takes about 4.3 s, and 300 tokens takes about 26 s.

Three consequences shape the whole design:

- **Output length matters more than prompt length.** Answers must be short and made of IDs, not sentences.
- **Every extra model call in a chain adds its own writing time.** Splitting one call into three calls in a row can make things slower, not faster. Calls should run side by side, not one after another, wherever the logic allows.

- **Writing speed is limited by memory bandwidth, not by the NPU.** The NPU is much faster than the CPU at reading a prompt, but not at writing an answer. So the CPU is a useful second lane for short-prompt, short-answer jobs.

Design rules

Eight rules, each answering one of the problems above or one of the current drawbacks.

1. **The tick never waits for a model.** The deterministic path runs at the start of every tick and publishes an assessment within 200 ms. Model results arrive later and update that assessment. This is what lets the CPU start tick $t+1$ while the NPU is still finishing tick t .
2. **Two model lanes, always loaded.** One `llama-server` process on the NPU and one on the CPU, started once and kept running. No model reload per call. One CPU core is kept for the deterministic path so a model running on the CPU can never delay it.
3. **Same model on both lanes.** One GGUF model file, run on the NPU through ggml-hexagon and on the CPU through llama.cpp's CPU backend. Any model job can then run on either lane, so the scheduler is free to move work, and answers don't change depending on where a job ran.
4. **Small calls.** Every prompt stays under 1280 tokens and every answer under about 60 tokens (the operator query agent is the one exception). This also means the fixed-size 1280-token compiled NPU models become usable later.
5. **Side by side, not in a chain.** At most two model calls in a row on the critical path: diagnose, then verify. Work that can run in parallel goes to the two lanes at the same time.
6. **Split work by when it is needed, not only by topic.** Some work happens every tick (checks), some when a new note arrives (reading it), some when the evidence changes (diagnosis), and some whenever a lane is free (operator questions). Only the first is on the clock every tick.
7. **Agents only talk through the blackboard.** The world model is the shared board. Each section of it has exactly one writer. No agent calls another agent directly.
8. **Code checks every model answer before it is used.** Cited IDs must exist, the answer must not be stale, and the schema must be valid. If not, the deterministic answer stands.

The agents

Ten agents: five are plain code that run every tick on a reserved CPU core, and five call the model only when something specific happens. The old diagnostician is split by plant side (water and heat) so the two halves can run at the same time on different units, and reading text is moved out of the diagnosis entirely.

Code agents (no model, reserved CPU core, every tick)

Agent	What it does	Replaces
1. Sensor agent	Reads the new samples, runs the five check families, writes facts with tick-stamped IDs (<code>t84.F1</code>), builds one signature for the water side and one for the heat side	L0, L1, L1b
2. Drift watcher	Keeps a running sum of small offsets in load-corrected bed temperature (and other slow relationships) over hours; raises a WATCH fact when the sum passes a threshold	New; targets slow fouling
3. Retriever and belief	Matches each side's signature against the case library, groups look-alike cases, lists candidate causes from the plant graph, updates belief scores (with the direction fix and pruning), and folds model answers into belief when they arrive	L3 + belief update
4. Scheduler	Decides which model jobs to create this tick, gives each a priority and a deadline, picks a lane, cancels or replaces stale jobs	L2 triage + orchestrator
5. Gate and memory	Checks every answer (citations, staleness, schema), picks actions from the catalogue, decides escalation, publishes the assessment, writes the world model	L6, L7

Model agents (event-driven, on the NPU or CPU lane)

Agent	Reads	Writes	Runs when	Default lane	Prompt / answer (tokens)
6. Water diagnostician	Water-side facts, water-side note-facts, the look-alike group's cases in compact form, one line about the heat side	Case group, top 3 case IDs with confidence and cited fact IDs, the ID of the check that would separate them	Water-side evidence changes and is at WATCH or above	NPU	~700 / ≤60
7. Heat diagnostician	Same, for the heat side (bed temperature, pressure, steam temperature, with steam flow as load)	Same	Heat-side evidence changes and is at WATCH or above	CPU (NPU if free)	~700 / ≤60
8. Verifier	The merged claims and all current facts; no cases, no notes	Pass or fail per claim, and the fact ID of the strongest contradiction	After a merge, when top confidence is in the uncertain band or the level is INVESTIGATE or above	Whichever lane is free first	~350 / ≤30
9. Text reader	One new engineer note or record, plus the tag and equipment dictionary	A note-fact (N3): tags, equipment, what it claims, when, reliability	A note or record arrives	CPU	~400 / ≤50
10. Query agent	The operator's question and the current assessment	A short answer citing fact IDs, flagging a wrong premise	The operator asks something; only when a lane is idle	CPU	~600 / ≤120

Why split by plant side. Most faults show up on one side only: feed-path faults on the water side, fuel and bed faults on the heat side. In those ticks only one diagnostician runs, as a single short call. When both sides move (a tube leak lowers drum level and also cools the bed), both run at the same time, one per lane, and the belief step merges them. Neither waits for the other.

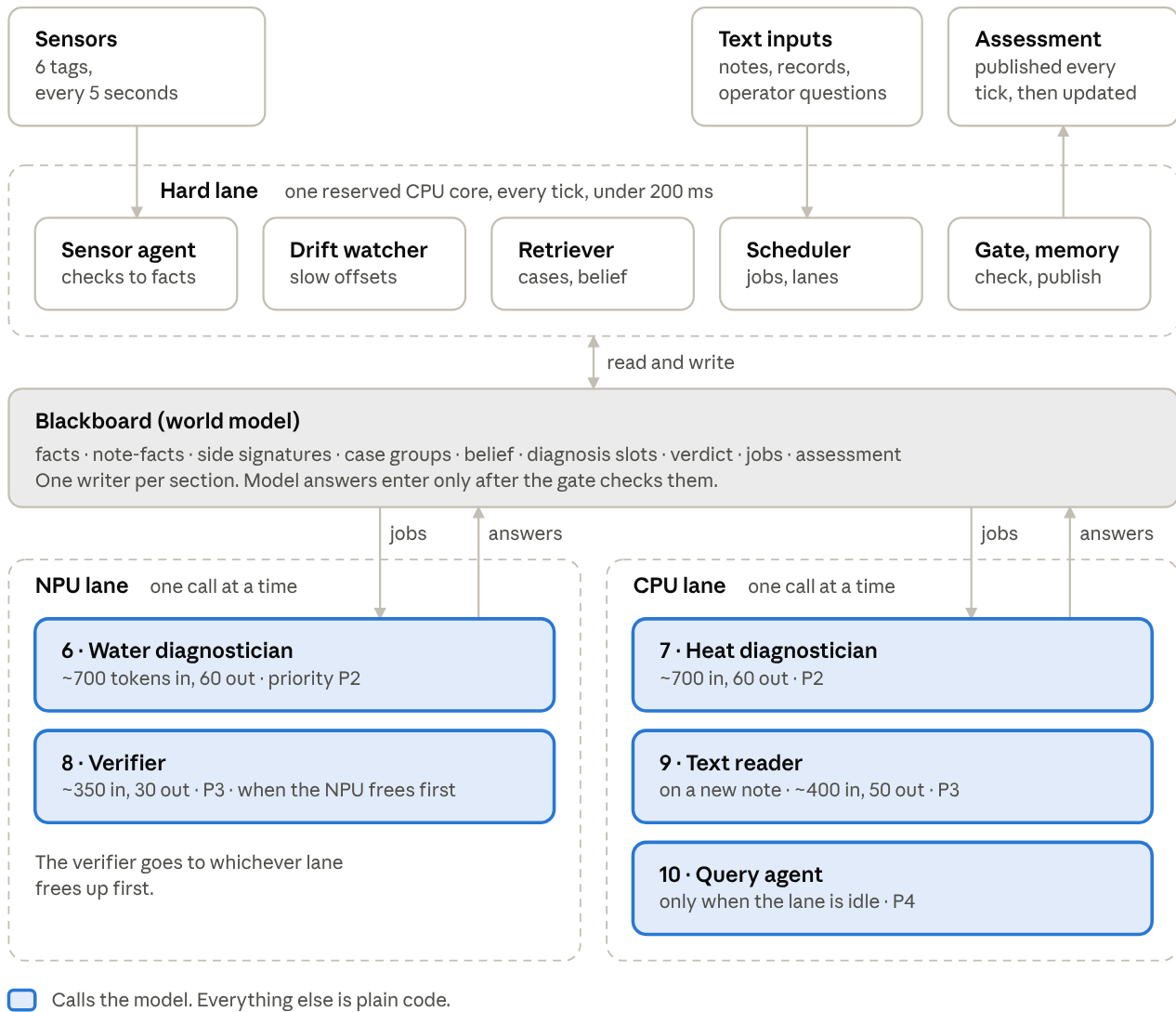
Why no model agent runs on every tick. A model call per subsystem per tick would add a chain of writing time to every tick and fill both lanes with repeated work. Facts, signatures and belief are already computed by code every tick. The model is called only when that evidence changes.

Why text is read separately. Notes arrive about ten times per episode. Reading each one once, when it arrives, and storing the result as a short note-fact keeps raw prose out of every diagnosis prompt. That alone removes a large share of today's ~2484 tokens.

Architecture

The system has three lanes that work at the same time and one shared blackboard between them. The hard lane is plain code on one reserved CPU core. The two model lanes each run one model call at a time, one on the NPU and one on the rest of the CPU.

Code runs every tick; model calls run on two lanes at once



architecture · hard lane, blackboard, two model lanes

Read it top to bottom. Sensor samples and text arrive at the top. The hard lane turns them into facts, belief and jobs every tick. Jobs go down to whichever model lane the scheduler picks, and answers come back up to the blackboard, where the gate checks them before they change anything. The assessment is published from the hard lane every tick, whether or not a model answer has arrived.

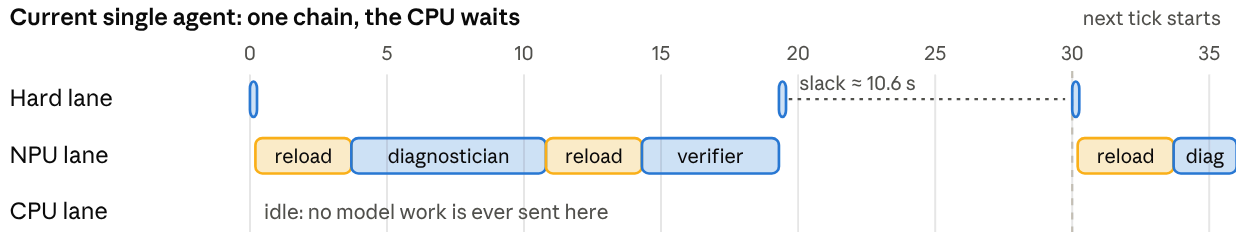
Execution timeline: critical path and slack

On a fault tick where both sides of the plant move, a verified diagnosis is ready at about 10 s instead of about 19 s. That roughly doubles the slack before the next tick, and the CPU does real model work instead of waiting.

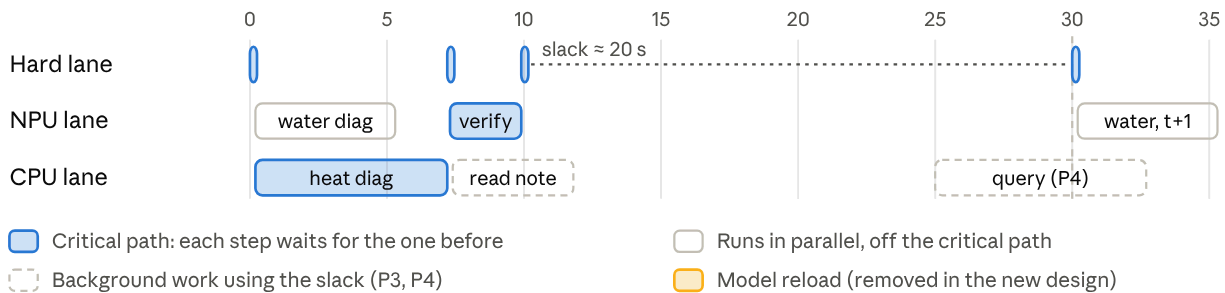
Splitting the work halves the time to a verified answer and doubles the slack

One fault tick where both plant sides move. Seconds from the tick start; planning estimates from measured rates.

Current single agent: one chain, the CPU waits



New design: two lanes work at once



one fault tick, before and after · critical path and slack

In simple words

Critical path is the chain of steps where each one has to wait for the one before it. Its length is the earliest moment a verified answer can exist. Speeding up anything *not* on this chain does not make the answer arrive sooner.

- **Before:** checks → reload → diagnostician → reload → verifier → gate. Everything is on one chain on one unit, so the whole 19.4 s is critical path.
- **After:** checks → heat diagnostician (on the CPU) → verifier (on the NPU) → gate, about 10 s. The water diagnostician runs on the NPU at the same time as the heat one and finishes first, so it is not on the critical path. The heat side is the slower of the two, so it sets the pace.

Slack is time left before the deadline after the critical path finishes. It is room to absorb a slow call, a hot chip, or an urgent job.

- **Before:** about 10.6 s of slack on the tick, and the CPU lane is idle for the whole 30 s.

- **After:** about 20 s of slack. The scheduler fills idle lane time with work that is not urgent: reading a new note at about 7 s, and answering an operator question from 25 s. That question runs past the 30 s boundary on the CPU while the NPU is already working on the next tick's water side. This is the "one unit on one tick, the other unit on the next tick" overlap.

Two things to notice:

- The NPU sits idle for about 2 s (5.3 s to 7.3 s) waiting for the heat side, because the verifier needs both answers. Giving the heat side to the NPU instead would make things worse (about 10.2 s), since the NPU would have to do both sides one after the other.
- The hard lane is never in the way. It runs its few milliseconds at the start of each tick on its own core, whatever the model lanes are doing.

On a tick where only one side moves (the common case), there is one diagnostician call and one verifier call: about $0.2 + 5.1 + 2.6 \approx 8$ s on the NPU, with the CPU lane free for notes and questions. **On a steady fault tick** where nothing changed, there are no model calls at all; the belief carries forward and the tick costs only the hard lane.

Scheduler

The scheduler is plain code that runs on the reserved core. Each tick it decides what model work is needed, how urgent it is, and which lane should do it. Its job is to keep both lanes busy with useful work and never let a model call delay the tick.

Three lanes

Lane	Runs	One at a time?	Notes
Hard lane	Code agents 1-5	Yes, in order	One CPU core reserved. Must finish within 200 ms every tick
NPU lane	Model calls via llama-server on HTP0	Yes	Fast at reading prompts; writing speed limited by memory
CPU lane	Model calls via llama-server on the CPU backend	Yes	Fixed thread count on the remaining cores, never on the reserved core

The NPU lane also needs a small host thread on the CPU to feed it. That thread is kept off the reserved core too.

Priorities and deadlines

Priority	Work	Deadline	Rule
P0	Hard path (code agents 1–5)	200 ms, every tick	Never queued behind anything
P1	Urgent diagnosis (trip predicted within 5 min, or a CRITICAL fact)	10 s	Goes to the front of the faster lane; shortest prompt; verifier skipped
P2	Diagnosis when a side's evidence changes	30 s after the evidence tick	Normal case
P3	Verifier; text reader	60 s	Text reader moves up to P2 if its note mentions a tag with an open finding
P4	Operator query, experience-store promotion	None	Only runs on an idle lane; dropped first

How a lane is chosen

For each lane the scheduler predicts when the job would finish:

$$t_{\text{finish}} = t_{\text{lane free}} + \frac{\text{prompt tokens}}{\text{prefill rate}} + \frac{\text{max answer tokens}}{\text{decode rate}}$$

It picks the lane with the earliest finish. The rates are not fixed constants: they are updated after every call from the measured timings, so the choice follows the real state of the chip, including heat throttling.

Working across ticks

- **Tick $t+1$ starts on time.** At each 30 s boundary the hard lane runs, whatever the model lanes are doing. A diagnosis for tick t that is still running continues.
- **Every result is stamped with its evidence tick.** The gate accepts it if it is at most one tick old, or if that side's evidence hasn't changed since. Otherwise it is dropped and logged as stale.

- **No backlog.** At most one waiting job per (agent, side). A newer job replaces an older waiting one. A running job finishes, unless it is P4 or superseded and the runtime can stop it early.
- **Only call the model when something changed.** A side's diagnosis is requested only if its signature, its open findings or its note-facts changed since the last one. During a steady fault, most ticks need no model call; the belief simply carries forward.
- **Bounded waiting.** Calls are not interrupted mid-way, but every answer is capped short, so an urgent job waits at most about one short call (a few seconds) for a lane.

Policies to compare in the scheduling study

Policy	Rule	What it tests
Fixed placement	Water diagnostician on NPU, heat on CPU, verifier on CPU	Simple baseline
Earliest finish	The prediction above, per job	Whether adapting to load helps
Tick alternation	All model work for odd ticks on the NPU, for even ticks on the CPU	Your "one unit per tick" idea. Doubles capacity, but answers can take up to 60 s and differ by lane
Priority-reserved	Like earliest finish, but the NPU is kept free when a trip is predicted	Mixed-criticality: guaranteed response for urgent work

Degradation ladder

Under heat or energy pressure, step down in this order. The deterministic answer is never affected.

Rung	Change	Expected cost
0	Full design	Baseline
1	Verifier only in the uncertain band	Small drop in faithfulness
2	Verifier off; text reader delayed to idle time	Notes-dependent episodes slower to resolve
3	One lane only (the cooler one); P4 work dropped	Longer diagnosis time

Rung	Change	Expected cost
4	Only the side with the more severe facts is diagnosed	Cross-side faults weaker
5	Model off; deterministic ranking and rule-based actions	No ranked explanation, but detection, alarms and safe actions still work

Blackboard and messages

Agents share one blackboard (the world model) instead of sending each other messages. Results arrive at different times from two lanes, and a shared board gives the gate one place to check what is current. The only messages are jobs going from the scheduler to a lane, and results coming back. This section is Track 2's area and should be confirmed or changed by that track.

Sections and who writes them

Each section has one writer. Anything else that wants to change it has to go through that writer.

Section	Written by	Read by
Facts ($t_{84.F1}$...; drift facts $t_{84.D1}$...)	Sensor agent; drift watcher writes only its own D IDs	Everyone
Note-facts ($N1$...)	Text reader (after the gate checks them)	Diagnosticians, query agent
Side signatures (water, heat)	Sensor agent	Retriever, scheduler
Case groups and candidates	Retriever and belief	Diagnosticians
Belief (causes with log-odds)	Retriever and belief only	Scheduler, gate, verifier
Water diagnosis slot, heat diagnosis slot	Gate, after checking each diagnostician's answer	Retriever and belief

Section	Written by	Read by
Verdict	Gate, after checking the verifier's answer	Gate, belief
Job table	Scheduler	Lanes, telemetry
Assessment and event log	Gate and memory	The engineer, the evaluator

Model answers never write to belief directly. The belief agent takes a checked answer as evidence and moves each case's log-odds by a capped amount. This keeps the lesson from the old `_merge` bug: a model answer nudges belief, it never replaces it.

Facts need tick-stamped IDs

Today fact IDs restart every tick (`F1` , `F2` ...). That is fine when the model answers inside the same tick, but here an answer can arrive one tick later, and `F1` would then mean a different fact. IDs therefore carry their tick: `t84.F1` . The gate checks citations against the facts of the answer's evidence tick.

Message formats

Job (scheduler to lane):

```
{ "job": "j-0841", "agent": "diag_water", "evidence_tick": 84,
  "priority": 2, "deadline_s": 30, "lane": "npu",
  "prompt_tokens": 690, "max_answer_tokens": 60 }
```

Diagnosis answer (what the model writes; short keys and IDs only, about 50 tokens):

```
{ "g": "W",
  "r": [ [ "RCA-01", 0.6, [ "t84.F1", "t84.F2" ] ], [ "RCA-16", 0.3, [ "t84.F2" ] ] ],
  "sep": "RCA-01", "n": [ "N3" ], "x": [ ] }
```

`g` is the look-alike group, `r` the ranked cases with confidence and cited facts, `sep` the case whose discriminating check should be shown to the engineer, `n` the note-facts used, `x` facts it could not explain. Code turns this into the sentences the engineer reads.

Result envelope (lane to board): the existing envelope, plus `evidence_tick` , `lane` , and `queue_wait_ms` , so the scheduling study can see how long each job waited and where it ran.

Failure handling

What goes wrong	What happens
A call misses its deadline	Result dropped when it arrives; deterministic answer stands; deadline miss logged for that priority
Answer is not valid JSON	One repair call, only if it can still finish before the deadline; otherwise dropped
A cited fact ID doesn't exist	The citation is removed; if under half are real, the whole answer is dropped (same rule as today)
Answer is stale (evidence changed since)	Dropped, logged as stale; a new job has already been queued by the scheduler
The two diagnosticians disagree	Both go into belief as separate evidence; the verifier sees both; any fact neither explains is listed as unexplained
A lane's server crashes	Restarted in the background; its jobs move to the other lane; ladder rung 3 until it is back
The text reader misreads a note	Note-facts carry a reliability score; sensor facts always win when they conflict

Time budgets

With the rates we have measured so far, a fault tick should finish its verified diagnosis in about 8–10 s instead of about 19 s, leaving about 20 s of slack before the next tick. These are planning numbers: the two lanes were measured with different models, so both must be re-measured once one model runs on both.

Rates used for planning

Lane	What was measured	Prefill (tok/s)	Decode (tok/s)
NPU, llama.cpp	Llama 3.2 3B Q4_0, ~1150-token prompt	~909	~13.9

Lane	What was measured	Prefill (tok/s)	Decode (tok/s)
CPU	Gemma 3 1B, LiteRT-LM	~126	~42
NPU, compiled graph (later option)	QAIRT context binary, ~1150-token prompt	~1953	~23.5

The compiled-graph row only becomes usable because every prompt in this design fits under 1280 tokens.

Token budget per agent

Part of the prompt	Water or heat diagnostician	Verifier
Rules and answer format	~180	~120
This side's facts (up to 8 lines)	~120	all current facts, ~170
Note-facts for this side (up to 4)	~80	—
Look-alike group cases, compact (up to 4)	~240	—
One line about the other side	~30	—
Current belief, top 3	~50	—
Claims to check	—	~60
Total prompt	~700	~350
Answer cap	60	30

Estimated time per call

Call	Lane	Reading the prompt	Writing the answer	Total
Water diagnostician	NPU	$700 \div 909 \approx 0.8 \text{ s}$	$60 \div 13.9 \approx 4.3 \text{ s}$	~5.1 s
Heat diagnostician	CPU	$700 \div 126 \approx 5.6 \text{ s}$	$60 \div 42 \approx 1.4 \text{ s}$	~7.0 s

Call	Lane	Reading the prompt	Writing the answer	Total
Verifier	NPU	$350 \div 909 \approx 0.4 \text{ s}$	$30 \div 13.9 \approx 2.2 \text{ s}$	$\sim 2.6 \text{ s}$
Verifier	CPU	$350 \div 126 \approx 2.8 \text{ s}$	$30 \div 42 \approx 0.7 \text{ s}$	$\sim 3.5 \text{ s}$
Text reader	CPU	$400 \div 126 \approx 3.2 \text{ s}$	$50 \div 42 \approx 1.2 \text{ s}$	$\sim 4.4 \text{ s}$
Query agent	CPU	$600 \div 126 \approx 4.8 \text{ s}$	$120 \div 42 \approx 2.9 \text{ s}$	$\sim 7.7 \text{ s}$

Two things stand out. On the NPU, writing the answer is most of the time, which is why answers are capped at 60 tokens. On the CPU, reading the prompt is most of the time, which is why the CPU suits short prompts. The heat diagnostician on the CPU (7.0 s) still beats waiting for the NPU to finish the water side first ($5.1 + 5.1 = 10.2 \text{ s}$).

Lane use on a fault tick where both sides move

	Current single agent	New design
Time to a verified diagnosis	$\sim 19 \text{ s}$ (including $\sim 7 \text{ s}$ of model reloads)	$\sim 10 \text{ s}$
NPU busy	$\sim 19 \text{ s}$ of 30	$\sim 7.7 \text{ s}$ of 30
CPU busy	$\sim 0.2 \text{ s}$ of 30 (under 1%)	$\sim 7.2 \text{ s}$ of 30 from diagnosis, plus text reading and queries in the slack
Slack before next tick	$\sim 11 \text{ s}$	$\sim 20 \text{ s}$

The NPU is busy for less time not because it does less useful work, but because the reloads are gone, the prompts are a third of the size, and the answers are short.

Which drawbacks this design fixes

The architecture fixes the speed and structure problems. It does not fix the correctness problems in the belief update, the metric or the data. Those must be fixed first in the single agent, so the multi-agent version has a trustworthy baseline to beat.

Drawback	In this design	How
CPU idle ~99% of the tick	Fixed	Second model lane on the CPU; both lanes run at once
Prompt too long for the fast NPU paths	Fixed	Every prompt under 1280 tokens; text read once by the text reader; compact case lines
Model reloaded on every call	Fixed	Persistent <code>llama-server</code> on each lane is a requirement of the design
Long answers can break the tick	Fixed	ID-only answers capped at 30-60 tokens; sentences written by code
One prompt does everything	Fixed	Ten agents with single jobs
Model comparisons not like-for-like	Fixed	Same model file on both lanes is a design rule
Verifier never disagrees	Partly	Narrower inputs and freedom to run on a different lane; still needs a test with a known-wrong answer
Slow fouling not detected	Partly	Drift watcher adds a cumulative-offset check; threshold still has to be tuned on normal episodes
Belief update ignores direction; no pruning	Not by design — fix first	Code fix in the belief agent; the design assumes it is done
Scoring counts exact top-1 only	Not by design — fix first	Group-level metric in the evaluator
Hold-out may test nothing; coverage tool disagrees	Not by design — fix first	Data and evaluation work
Action precision 0.38	Not by design	Improves when ranking improves

Drawback	In this design	How
No GPU or energy numbers	Not by design	Measurement work; the GPU can be added as a third lane later
Look-alike cases; synthetic data; drum constant K	Cannot be fixed by any architecture	Reported as limits; advisor questions

Of the 14 rows above, the design fixes 6, partly fixes 2, and leaves 6 to separate work.

Measurement

All targets from the Plan of Action stay: Q1–Q7, S1–S7, constraints C1–C6, and energy per correct diagnosis. The multi-agent design is only a result if it beats the fixed single agent on these, with the same model and the same episodes.

New measurements this design needs

Measure	Definition	Why
Diagnosis latency	Time from the evidence tick to a verified answer on the board	The main speed claim
Lane busy time	Share of each tick that the NPU lane and the CPU lane are working	Shows whether the idle-CPU problem is solved
Queue wait per priority	How long jobs wait for a lane, split by P1–P4	Shows whether urgent work is protected
Deadline misses per priority	P0 must stay at exactly zero; P1 has a target; P2–P4 are reported	Mixed-criticality claim
Stale-drop rate	Share of model answers thrown away because the evidence moved on	Cost of working across ticks
Shared-memory slowdown	Each lane's decode speed when the other lane is also decoding, compared with alone	Both lanes read from the same RAM; this is the main unknown

Measure	Definition	Why
Model calls per tick	Average and maximum	Effect of calling the model only on change

Two harness modes

- **Lockstep mode, for accuracy.** The harness waits until every job for a tick has finished before moving to the next tick. Results don't depend on timing, and it can run on the laptop with any backend. Use it for Q1-Q7 on all 30 episodes.
- **Real-time mode, for system numbers.** On the board, ticks fire every 30 s of wall-clock time and jobs really overlap. An episode takes its full length (about 55 minutes), so run a subset: one episode per fault family plus two normal ones is about 7 hours. Use it for latency, lane use, deadline misses, stale drops, temperature and energy.

Running accuracy in both modes on the same subset shows what working across ticks costs: if real-time accuracy is clearly lower than lockstep accuracy, too many answers are arriving stale.

The comparisons that make the result

1. **Single agent vs multi-agent**, after the Phase 0 fixes, same model: accuracy, diagnosis latency, energy per correct diagnosis.
2. **Scheduling policies**: fixed placement, earliest finish, tick alternation, priority-reserved.
3. **Structure ablation**: code only → one side's diagnostician → both sides → plus verifier → plus text reader. Shows what each agent adds.
4. **Accuracy vs energy**: one point per configuration, the Pareto plot from the original plan.

New risks

The biggest unknown is whether two lanes writing answers at the same time slow each other down, because both read model weights from the same RAM. The rest are manageable with the checks below.

Risk	Why it could happen	What we do
Both lanes slow down when they decode together	Decode on both units is limited by the same LPDDR5 bandwidth	Measure decode speed alone vs together first. If the slowdown is large, the scheduler staggers jobs so their writing phases overlap less
Splitting by side misses faults that show on both sides	A tube leak lowers drum level and also cools the bed	Each side's prompt carries one line about the other side; belief merges both; verifier sees both. If accuracy on cross-side faults drops, add one joint call (under ~1100 tokens) for look-alike groups that span both sides
One small model on both lanes is less accurate than a larger one on the NPU alone	The same-model rule pushes toward a model small enough for the CPU	Choose the model on accuracy, citation faithfulness and broken-JSON rate, not only speed
Answers arrive stale too often	Evidence changes faster than the model answers	Stale-drop rate is measured; if high, shorten prompts further or raise P2's priority
The CPU lane heats the chip and slows the NPU	Both units share one package and one thermal limit	Log temperature on every run; the ladder drops to one lane when hot
The text reader misreads a note	Small models misread shorthand	Note-facts keep a reliability score and a link to the raw note; sensor facts win on conflict
Two copies of the model in memory	One copy is repacked for the NPU, one used by the CPU	About 2× the model size; fine on 16 GB for a 3B Q4_0 model or smaller. Track peak memory (S5)

Risk	Why it could happen	What we do
More moving parts, more silent bugs	The DEGRADED latch and <code>_merge</code> bugs came from exactly this kind of hand-off	A contract test per agent, mutation checks on each, and the event log recording every job and drop

Build order

Each phase changes one thing and has a check that must pass before the next starts, so any change in the numbers can be traced to one cause. Phase 0 fixes the single agent first, because the multi-agent version is only a result if it beats a trustworthy baseline.

Phase	What gets built	Check before moving on
0. Fix the single agent	Belief update compares direction and speed; hypothesis pruning; group-level metric in the evaluator; answer length capped; persistent <code>llama-server</code> ; one model chosen for both lanes	New baseline on all 30 episodes, mock and NPU, in lockstep mode
1. Plumbing only	Tick-stamped fact IDs, blackboard sections with single writers, job table, both lanes running, scheduler with fixed placement — still calling the old single diagnostician	Accuracy unchanged from Phase 0 (only plumbing changed); zero hard-path misses; lane telemetry recorded
2. Shrink the prompt	Text reader and note-facts; compact case lines; ID-only answers	Diagnostician prompt under 1280 tokens; any accuracy drop is within a limit the team agrees before running
3. Split by side	Water and heat diagnosticians in parallel, belief merge, verifier on whichever lane is free	Group-level accuracy at least Phase 2's, checked separately on cross-side faults; diagnosis latency measured

Phase	What gets built	Check before moving on
4. Real time on the board	Real-time harness mode, drift watcher, shared-memory slowdown test	Zero hard-path misses across the subset; stale-drop rate and lane use recorded; check whether slow-fouling episodes are now flagged
5. Scheduling study	Four policies, degradation ladder, power measurement	Accuracy vs energy plot, one point per configuration

Track 2 (agent communication and state) owns the blackboard sections, the message formats and the failure handling, which land in Phase 1 and are exercised in every phase after.

Open decisions

- ☐ **Which model runs on both lanes?** It must be small enough for useful CPU speed and good enough for diagnosis. Decide on accuracy, faithfulness and broken-JSON rate from a short test on the fault episodes.
- ☐ **Blackboard or direct messages?** This plan uses a blackboard with single writers. Track 2 should confirm this or argue for message passing, using the failure cases in the Blackboard section as the test.
- ☐ **Late answers.** Is an answer that arrives one tick late acceptable to report as that tick's diagnosis, or must every number be scored against the tick it was delivered in? This changes how Q2 and lead time are computed in real-time mode.
- ☐ **Allowed accuracy drop when shrinking the prompt** (Phase 2 check). Agree the limit before running, so it isn't chosen after seeing the result.
- ☐ **Is the verifier worth keeping?** Decide after the known-wrong-answer test; it costs a call on every uncertain tick.
- ☐ **Advisor:** is reporting "verified diagnosis within ~10 s, deterministic assessment every tick" an acceptable way to meet the 30 s tick deadline, instead of finishing all model work inside each tick?